

Project Contract

PayTrack

TUW Racing Team Project Group

March 25, 2026

Contents

1	Project Name	3
2	Team Members	3
3	Initial situation & existing similar products	3
4	Project Description	4
5	Target Audience	4
6	Asset Architecture & Strategy	5
6.1	Asset Overview	5
6.2	Asset Contribution	6
6.3	Gap and Risk Analysis	6
7	Functional Requirements	7
7.1	Epic 1: Authentication & User Management (10%)	7
7.2	Epic 2: Invoice Submission (10%)	9
7.3	Epic 3: Finance Team Workflow (15%)	11
7.4	Epic 4: Payment Requests (Members owe money) (10%)	13
7.5	Epic 5: Import & Automation (10%)	15
7.6	Epic 6: Notifications (5%)	17
7.7	Epic 7: Budget & Finance Overview (15%)	17
7.8	Epic 8: Data Export & Archiving (5%)	20
7.9	Epic 9: Progressive Web App (20%)	21
8	Non-Functional Requirements	25
9	Domain Model	31
10	Working Structure & Simple Project Plan	31
10.1	Distribution of Roles & Role Responsibilities	31
10.2	Project plan - PayTrack	32
10.3	Milestones	33

11 Project Boundaries	33
11.1 In Scope	33
11.2 Out of Scope	34
11.3 Assumptions and Constraints	35
12 Supply Components (Deliverables)	35
12.1 Software Artifacts	35
12.2 Management Artifacts	36
12.3 Out-of-scope Artifacts	36
13 Risk Assessment	36
13.1 Risk Register	36
13.2 Most Critical Risks	38
13.3 Risk Handling Strategy	39
13.4 Risk Matrix	39
14 Information Process	40
14.1 Communication	40
14.2 Documentation	40
15 Special Features	41

1 Project Name

Working Title: PayTrack

PayTrack is a centralized platform for the TUW Racing Team to submit invoices and manage financial workflows, enabling structured, transparent, and efficient payment processing.

2 Team Members

Name	Student ID	Role	Backup Role
Jonas Tatzberger	12002213	Team Coordinator	Documentation
Thimon Pelka	12123690	Technical Architect	DevOps
Michael Lohwasser	12225130	DevOps	Technical Architect
Christoph Bräuer	12223394	Test Lead	AI Expert
Jakob Jeschke	12322939	UI Expert	Test Lead
Jakob Schreiblehner	12225107	AI Expert	Documentation

3 Initial situation & existing similar products

The TUW Racing Team currently manages financial processes like invoice submission, refunding, and payment tracking using a combination of informal tools, including email, messaging platforms (e.g. Slack), and spreadsheets. Team members submit expenses manually, often by sending messages or uploading files in an unstructured way. The finance team then processes these requests manually, keeping track of statuses, payments, and budget allocation across multiple disconnected tools. This leads to a fragmented and inefficient workflow with no central system of record.

Several existing solutions partially address financial workflow management but do not fully meet the specific needs of the TUW Racing Team. General-purpose tools such as *Google Sheets* and *Microsoft Excel* are widely used for tracking expenses and budgets, but they rely heavily on manual input and lack structured workflows, validation mechanisms, and automation. Communication platforms like *Slack* support coordination but are not designed for persistent financial data management, leading to information loss and poor traceability.

More specialized expense management tools such as *Expensify* or *Splitwise* provide structured workflows and automation features. However, these solutions are typically designed for corporate environments or private group expense sharing and do not align well with the racing team's specific requirements, such as cost centre handling, integration with existing tools (e.g., Google Drive), and flexible workflows for both refunding and payment requests.

In addition to lightweight tools, full-scale accounting solutions such as *QuickBooks* and *Xero* exist for managing financial processes. These systems offer comprehensive functionality, including bookkeeping, tax handling, and advanced financial reporting. However, they are primarily designed for businesses with formal accounting requirements and introduce a high level of complexity. For the TUW Racing Team, such solutions would be

excessive, as they include many features that are not needed for simple invoice submission and refunding workflows. Their setup, maintenance, and required domain knowledge make them unsuitable for a lightweight, compact and focused solution like **PayTrack**.

4 Project Description

The PayTrack project aims to design and implement a centralized web-based application that supports the TUW Racing Team in managing invoice submissions, refunding processes, and internal payment requests. The system introduces structured workflows for both team members and the finance team, reducing manual effort and improving transparency across all financial activities. By integrating features such as automated validation, status tracking, and data consistency, PayTrack enhances efficiency and provides a reliable foundation for financial management within the team.

From the perspective of a regular team member, PayTrack provides a simple and intuitive way to submit invoices, upload receipts, and track reimbursement status. Instead of relying on scattered communication channels, users can follow a clear and structured process with immediate feedback on whether their submissions are approved, pending, or paid. The system reduces uncertainty and effort by offering a centralized, transparent interface for all personal financial interactions with the team.

As a finance team member, PayTrack enables efficient management of all financial workflows in a centralized system. Submitted invoices can be reviewed, validated, and processed through a structured workflow with clear status tracking and history. The system supports better oversight of budgets, payments, and outstanding requests across different teams and cost centres while reducing manual coordination and administrative effort. This leads to improved accuracy, transparency, and control over the team's financial operations.

5 Target Audience

The primary target audience of PayTrack consists of finance team members of the TUW Racing Team, who are responsible for overseeing invoice submission, validation, and payment processes. They ensure that all incoming invoices are complete, compliant, and processed in a timely manner, while maintaining oversight of budgets, cash flow, and spending across categories such as equipment, travel, and operations. Their expectations are measurable: reducing incomplete or incorrect submissions, accelerating approval and payout cycles, and providing clear visibility into financial data and reporting.

The secondary target audience includes regular team members who submit expenses, pay fees, and request reimbursements. Currently relying on fragmented tools such as email, spreadsheets, and messaging platforms, they require a simple, fast, and intuitive solution. For them, ease of use and transparency are critical—submitting invoices, uploading receipts, and tracking payment status must be more efficient than existing informal processes. Their consistent adoption is essential to ensure complete and reliable financial data and to enable the overall workflow to function effectively.

6 Asset Architecture & Strategy

The architecture of the PayTrack system is modeled as an Asset Network to identify the minimal set of assets.

6.1 Asset Overview

The PayTrack ecosystem is structured using the core Production Asset Network categories: Product, Process and Resource.

Resources: Human or technical agents that execute tasks and control the flow of data.

- Admin / Finance Team (Actor): The primary stakeholder responsible for validation, managing and budget oversight.
- Regular Team Member (Actor): Submits expenses and tracks repayment.
- PayTrack Application (System): The central Coordination Asset Network that connects conditions to asset properties and data sources.
- IronOCR or similar (Systems): The automated text-extraction engine used to pull metadata from uploaded receipt images.
- PostgreSQL Database (System): The primary relational database ensuring data integrity and persistent storage for all financial transactions and user states.
- Google Workspace (Systems): An external module providing user authentication (Google Login) and secondary persistent storage (Google Drive).
- Slack & Email (Systems): External modules providing asynchronous communication for user notifications, reminders and change requests.

Products: Data, artifacts or materials that are created, consumed or transformed by the system.

- Input Products: Uploaded Invoices, Receipt Images, imported Bank Statements, and Target Budget Configurations.
- Output Products: Finalized Payment Requests, matched Transaction Records, and accumulated Financial Reports.

Processes: Workflows, activities or sequence of steps that use resources to transform input products into output products.

- Invoice Lifecycle Management: The core process transitioning an invoice (Submitted -> Changes Requested -> Approved -> Paid).
- Automated Data Extraction & Matching: The process of using IronOCR or similar to read receipt data and automatically matching uploaded bank statements to internal records.
- Payment Request: The workflow where the Finance Team issues and tracks payment requests for members who owe the team money.
- Budgeting & Reporting: The process of accumulating active transaction data to provide real-time budget overviews for specific cost centres.

6.2 Asset Contribution

The solution relies on integration the new application with the existing shared environments where stakeholders currently access data.

- Paytrack Core & PostgreSQL (New Assets): Replaces the fragmented assets (Google Forms, scattered spreadsheets) by providing a unified website for data entry for lifecycle tracking.
- IronOCR or similar (New Asset): Acts as a data-extracting tool. When a user uploads a receipt, IronOCR automatically attempts to extract asset properties (like amount or date) to reduce manual entry and improve data quality.
- Google Accounts & OAuth (Existing & New Asset): The team's use of Google Drive means that Google accounts are an existing resource. PayTrack introduces the Google OAuth API as a technical interface, to provide immediate, secure authentication without requiring a custom credential management system.
- Google Drive (Existing Asset): Acts as a redundant, secure storage layer for all uploaded receipt artifacts. Paytrack will automatically mirror files here to ensure data safety.
- Slack & Email (Existing Assets): Used for automated notifications and change requests, bridging the interface between the Paytrack process and user communication.

6.3 Gap and Risk Analysis

To bridge the gap from the as-is manual state to the desired PayTrack project, the impact of potential undesired conditions must be analyzed.

Bridging the Gap from Legacy Assets (High Effort / High Risk)

- The Gap: We need to move the team away from their implicit, undocumented habits (Google Drive, Forms and Slack) to a strict, automated system.
- Legacy Complexity: The old way of doing things increases complexity because the existing data is unstructured and scattered.
- The Risk: If the new UI is harder or slower to use than doing it manually, the team members will refuse to use it or will use it incorrectly. Which lead to missing financial data.
- The Effort: It requires significant effort to build a completely new, unified database (PostgreSQL) and design a frontend application (PWA) that is so intuitive and user-friendly that it successfully convinces users to abandon their old habits.

Third-Party Sevices and External Dependencies (Medium Effort / Medium Risk)

- The Gap: The project rely heavily on external services (Google Drive, Google OAuth, Slack, Email and IronOCR).
- The Risk: These services are completely outside of our control. If Google changes its API rules, limits our usage, or temporarily goes down, our application will already fail, because logging into the application won't work without Google.

- **The Effort:** It takes medium effort to build fallback mechanisms. For example, the log-in process should be easily switched to a classic user log-in with a credential management system.

Data Quality and OCR Validation (Low Effort / Medium Risk)

- **The Gap:** We want to automate data entry using IronOCR
- **The Risk:** OCR heavily depends on the user taking a good photo. If a receipt is blurry or crumpled the automated reading will fail.
- **The Effort:** It requires minimal effort to fix, as we need to ensure the system allows the Finance Team to manually edit or correct the data when the OCR fails.

7 Functional Requirements

The following Stories resolve around these Roles:

- **User:** A regular User using the application. This would be a normal Racing Team member
- **Finance Team Member:** This is also a Racing Team "Employee" but part of the finance team. This user has generally more rights.
- **Team Lead:** A Team lead is a regular user who has a bit more rights when it comes to viewing budgets and goals. Generally he is mostly a normal user though.

For the estimated time we assumed a total available time for the Epics & Stories of about 750 hours. (As the other 150 we will definitely need for the meetings, organizational matters (project proposal, project contract, etc.) and preparations for the MR & IR).

Each User-Story also already has at least one Person assigned to it. This is not 100% final as Developers might take over US from others if they are faster.

7.1 Epic 1: Authentication & User Management (10%)

Estimated Effort for this Epic: approx. 75 hours

US-1: Login with Google

Story

As a user, I want to log in using my Google account so that I can securely access the PayTrack system.

Acceptance Criteria

- Login only possible via Google OAuth
- If a user logs in for the first time a new account/user is created
- The system stores: name, email, google login reference
- The system should be designed in a way that Google as an authentication provider should be easily replaceable

Estimated Effort

20 hours

Assigned to

Thimon Pelka

US-2: Manage Personal Information

Story

As a user, I want to manage my personal information so that my payment and contact details are correct.

Acceptance Criteria

- Users can edit: name, email, bank account(s), preferred bank account

Estimated Effort

20 hours

Assigned to

Jakob Schreiblehner

US-3: Mandatory Bank Information

Story

As a new user, I must provide my bank information so that reimbursements can be processed easily.

Acceptance Criteria

- If no bank account exists the user is forced to create a new one
- A bank account includes IBAN, BIC and Account Holder

Estimated Effort

15 hours

Assigned to

Thimon Pelka

US-4: View and manage all Users

Story

As a finance team member I want to be able to view all Users and manage them.

Acceptance Criteria

- A finance team member should be able to view a list of all members in the application and filter them by username, email and team
- A finance team member should be able to change the status of a member by assigning them to a team or setting them active/inactive

Estimated Effort

20 hours

Assigned to

Jakob Schreiblehner

7.2 Epic 2: Invoice Submission (10%)

Estimated Effort for this Epic: approx. 75 hours

US-5: Submit Expense Invoice

Story

As a user, I want to submit an invoice for reimbursement so that I can get refunded for expenses made for the racing team.

Acceptance Criteria

- User must provide: invoice amount, invoice date, description/purpose, module/team, payout type (= self paid / not yet paid), receipt upload, optional comment
- A user must not (!) provide the cost centre. This will be done by the finance team later.
- The receipt has to be uploadable using drag-and-drop
- On Upload of the receipt the backend should try to extract important information (like amount or date) from the uploaded receipt
- If the payout type is self paid then the user also has to select which bank account the money should be transferred to (from his personal bank accounts in his profile settings)

Estimated Effort

35 hours

Assigned to

Christoph Bräuer

US-6: Detect Duplicate Invoices

Story

As a user, I want to be warned if a similar invoice already exists so that duplicate reimbursements are avoided.

Acceptance Criteria

- System checks similarity based on: amount, date, merchant, invoice number
- If one or multiple similar invoices are found a warning is shown
- After confirmation of the warning the user can still submit the invoice

Estimated Effort

25 hours

Assigned to

Christoph Bräuer

US-7: View Submitted Invoices

Story

As a user, I want to see all my submitted invoices so that I can track their status.

Acceptance Criteria

- A User can see their invoice list and filter them by status, date, amount, description/purpose, module/team, payout type
- Each invoice must be in one of the defined states (like submitted, changes requested, approved, paid, declined)

Estimated Effort

15 hours

Assigned to

Jakob Jeschke

7.3 Epic 3: Finance Team Workflow (15%)

Estimated Effort for this Epic: approx. 112.5 hours

US-8: Review Submitted Invoices

Story

As a finance team member, I want to review submitted invoices so that I can verify their correctness.

Acceptance Criteria

- A Finance Team Member can: See all invoices, filter them, open a more detailed invoice view, download receipt
- A Finance Team Member filter the invoices by status, date, amount, description/purpose, module/team, payout type AND cost centre (unlike the normal user)

Estimated Effort

30 hours

Assigned to

Thimon Pelka

US-9: Request Changes

Story

As a finance team member, I want to request changes on an invoice so that the user can correct mistakes.

Acceptance Criteria

- A Finance team member can change the status to "changes requested" and contact the user via slack or email
- If a user is contacted the status is automatically changed to "changes requested". This could be changed via settings
- All status changes must be stored as a history with date, an identification of the user who changed the status and an optional reason

Estimated Effort

35 hours

Assigned to

Thimon Pelka

US-10: Approve Invoice

Story

As a finance team member, I want to be able to approve an invoice so that it can be paid.

Acceptance Criteria

- A Finance team member must assign a cost centre before being apply to approve an invoice
- A Finance team member can change the status to "approved"
- All status changes must be stored as a history with date, an identification of the user who changed the status and an optional reason

Estimated Effort

15 hours

Assigned to

Thimon Pelka

US-11: Decline Invoice

Story

As a finance team member, I want to decline an invoice so that invalid expenses are rejected.

Acceptance Criteria

- A Finance team member must provide a reason for the rejection
- A Finance team member can change the status to "declined"
- All status changes must be stored as a history with date, an identification of the user who changed the status and an optional reason

Estimated Effort

15 hours

Assigned to

Jakob Schreiblehner

US-12: Mark Invoice as Paid

Story

As a finance team member, I want to mark an invoice as paid so that the system reflects completed reimbursements.

Acceptance Criteria

- Finance team must provide: payment reference / purpose
- Payment date

Estimated Effort

15 hours

Assigned to

Jakob Jeschke

7.4 Epic 4: Payment Requests (Members owe money) (10%)

Estimated Effort for this Epic: approx. 75 hours

US-13: Create Payment Request

Story

As a finance team member, I want to request payments from members so that the team can collect fees or reimbursements. (Examples: Membership Fee, Merchandise, Event Tickets)

Acceptance Criteria

- A Finance team member defines: User, Amount, Purpose, team, cost centre and Due Date
- The system stores who created the payment request and when

Estimated Effort

20 hours

Assigned to

Christoph Bräuer

US-14: View Open Payment Requests (User)

Story

As a user, I want to see my open payment requests so that I know what I still need to pay.

Acceptance Criteria

- A User can see a list of all his open payments
- For each payment a User can see: amount, due date, purpose, status
- A User can filter these payments by amount, due date, purpose and status

Estimated Effort

20 hours

Assigned to

Christoph Bräuer

US-15: View Open Payment Requests (Finance Team)

Story

As a finance team member, I want to see all open payment requests so that I know who and what still needs to be paid.

Acceptance Criteria

- A finance team member can see a list of all open payments
- For each payment a finance team member can see: amount, due date, purpose, status, team and cost centre
- A finance team member can filter these payments by amount, due date, purpose, status, team and cost centre
- A finance team member has the option to contact the User of a payment request using slack or email

Estimated Effort

20 hours

Assigned to

Jakob Jeschke

US-16: Approve Open Payment Requests

Story

As a finance team member, I want to be able to manually approve an open payment request when I know the payment has arrived.

Acceptance Criteria

- A finance team member can set the status of a payment request to "paid".
- All status changes must be stored as a history with date, an identification of the user who changed the status and an optional reason

Estimated Effort

15 hours

Assigned to

Jakob Jeschke

7.5 Epic 5: Import & Automation (10%)

Estimated Effort for this Epic: approx. 75 hours

US-17: Import Order Lists

Story

As a finance team member, I want to upload an order list so that payment requests can be created automatically.

Acceptance Criteria

- The system accepts a CSV input
- The system parses the list and matches users by name or email
- The system automatically creates new payment requests drafts
- The finance team member has to approve the draft before actually generating the payment requests

Estimated Effort

35 hours

Assigned to

Michael Lohwasser

US-18: Upload Bank Statements

Story

As a finance team member, I want to upload bank statements so that payments can be automatically matched.

Acceptance Criteria

- The system parses the transactions and matches payments using amount, name (and if set also purpose)
- If the system finds matches then each match gets their status updated in a draft
- After updating the finance team member should get an overview over all payment requests which would be changed and also all payments which could not be matched
- The finance team member can accept the changes and then they get automatically written to the database

Estimated Effort

40 hours

Assigned to

Thimon Pelka

7.6 Epic 6: Notifications (5%)

Estimated Effort for this Epic: approx. 37.5 hours

US-19: Notify Users of Status Changes

Story

As a user, I want to receive notifications when invoice status changes so that I know the current progress.

Acceptance Criteria

- On status change a user should be notified
- Standard Notifications about status changes should be sent to the User using EMail

Estimated Effort

37.5 hours

Assigned to

Jakob Schreiblehner

7.7 Epic 7: Budget & Finance Overview (15%)

Estimated Effort for this Epic: approx. 112.5 hours

US-20: Expense Overview

Story

As a finance team member, I want to see an overview of all expenses and incomes so that I can track spending.

Acceptance Criteria

- A finance team user should be able to view the expenses and incomes and filter them by team, cost centre and time period

Estimated Effort

30 hours

Assigned to

Jonas Tatzberger

US-21: Expense Overview (Team Lead)

Story

As a team lead, I want to be able to see an overview of all expenses and incomes of my (!) team so that I can track spending.

Acceptance Criteria

- A team lead should be able to view the expenses and incomes of his team and filter them by cost centre and time period

Estimated Effort

10 hours

Assigned to

Jonas Tatzberger

US-22: Budget Monitoring

Story

As a finance team member, I want to compare expenses to budget targets so that overspending is visible.

Acceptance Criteria

- A finance team user should be able to see whether a certain team or cost centre has spent more or less than their target budget so far

Estimated Effort

20 hours

Assigned to

Jonas Tatzberger

US-23: View all Teams

Story

As a finance team member, I want to be able to view all teams.

Acceptance Criteria

- A finance team user should be able to view all teams and filter them by name, description or budget

Estimated Effort

15 hours

Assigned to

Jonas Tatzberger

US-24: Managing Teams

Story

As a finance team member, I want to be able to create, edit and delete teams.

Acceptance Criteria

- A finance team member should be able to create a team by supplying a team name, description and optional target budgets for certain due dates.
- A finance team member should be able to update the settings of a team.
- A finance team member should be able to delete a team. This should show a warning and how many invoices, user, etc. are affected

Estimated Effort

15 hours

Assigned to

Jonas Tatzberger

US-25: View all Cost Centres

Story

As a finance team member, I want to be able to view all cost centres.

Acceptance Criteria

- A finance team user should be able to view all cost centres and filter them by name, description or budget

Estimated Effort

10 hours

Assigned to

Jonas Tatzberger

US-26: Managing Cost centres

Story

As a finance team member, I want to be able to create, edit and delete cost centres.

Acceptance Criteria

- A finance team member should be able to create a cost centres by supplying a cost centres name, description and optional target budgets for certain due dates.
- A finance team member should be able to update the settings of a cost centres.
- A finance team member should be able to delete a cost centres. This should show a warning and how many invoices, user, etc. are affected

Estimated Effort

10 hours

Assigned to

Jonas Tatzberger

7.8 Epic 8: Data Export & Archiving (5%)

Estimated Effort for this Epic: approx. 37.5 hours

US-27: Export Financial Data

Story

As a finance team member, I want to export financial data so that it can be used for accounting or reporting.

Acceptance Criteria

- A finance team member should be able to export financial data (expenses and incomes) using csv or pdf
- A finance team member should also be able to only export the financial data of a specific team, cost centre and/or within a certain time period

Estimated Effort

20 hours

Assigned to

Jakob Schreiblehner

US-28: Automatic Archiving (Double-Saving) of Data

Story

As a finance team member, I want invoices automatically archived in Google Drive so that financial documentation is secure. Example Storage Architecture (as request by racing team): 2026/March/<invoiceid>.pdf

Acceptance Criteria

- Each uploaded invoice should also be saved to the google drive storage of the racing team to have a secure and redundant data storage

Estimated Effort

10 hours

Assigned to

Jonas Tatzberger

7.9 Epic 9: Progressive Web App (20%)

Estimated Effort for this Epic: approx. 150 hours

US-29: Install application on mobile device

Story

As a User, I want to be able to install PayTrack on my mobile device from the browser so that I can access it like a native app.

Acceptance Criteria

- The application can be added to the home screen on supported devices.
- The installed application opens directly from the home screen icon.
- The installation process is clearly indicated to the user where supported.

Estimated Effort

15 hours

Assigned to

Jakob Jeschke

US-30: Offline availability of essential views

Story

As a User, I want essential parts of the application to remain available even with limited or no internet connection so that I can still access important information when connectivity is unstable.

Acceptance Criteria

- Previously loaded core views remain accessible offline.
- The system informs the user when live data cannot be refreshed.
- Offline availability does not expose unauthorized or outdated sensitive data without clear indication.

Estimated Effort

25 hours

Assigned to

Jakob Jeschke

US-31: Push notifications for status changes

Story

As a User, I want to receive push notifications about status changes of my submitted invoices and payment requests such as approval, rejection, or payment completion, so that I stay informed without opening the application repeatedly.

Acceptance Criteria

- Notifications are triggered for relevant workflow events.
- Users can enable or disable notifications where platform support exists.
- The system handles unsupported platforms gracefully.

Estimated Effort

35 hours

Assigned to

Michael Lohwasser

US-32: Mobile-friendly invoice upload

Story

As a User, I want to upload invoices and receipts easily from my mobile device so that I can submit expenses directly after a purchase.

Acceptance Criteria

- The upload flow is usable on smartphone screens.
- Users can select files from their device using supported browser functions.
- Required metadata can be entered comfortably on mobile devices.

Estimated Effort

15 hours

Assigned to

Michael Lohwasser

US-33: Platform-aware fallback behavior

Story

As a user on devices with platform-specific limitations, such as iOS, I want the system to provide a usable fallback if certain PWA features are not fully supported so that the application remains functional.

Acceptance Criteria

- Unsupported PWA features do not block core application usage.
- Alternative user flows are provided where full PWA support is unavailable.
- Platform-specific limitations are communicated clearly where relevant.

Estimated Effort

25 hours

Assigned to

Jakob Jeschke

US-34: Offline Upload

Story

As a user I want to upload invoice documents even when I am temporarily offline so that I can capture expenses immediately and submit them once the connection is restored.

Acceptance Criteria

- Users can select or capture invoice documents while offline.
- The document and related form data are stored locally on the device until an internet connection is available again.
- The system clearly indicates that the upload is pending and has not yet been synchronized.
- Once the connection is restored, the upload can be submitted automatically or after user confirmation.
- The user is informed whether the synchronization was successful or whether manual action is required.

Estimated Effort

35 hours

Assigned to

Michael Lohwasser

8 Non-Functional Requirements

The following non-functional requirements define the quality attributes of the PayTrack system. Each requirement includes a category, a description, and measurable acceptance criteria where applicable.

NFR-1: Performance

Description

The system shall provide responsive interactions for all core user actions such as viewing invoices, submitting data, and navigating between views.

Acceptance Criteria

- Under normal operating conditions, standard views (e.g. dashboard, invoice list) load within 2 seconds.
- Filtering and basic interactions respond within 2 seconds.
- Detailed views (e.g. invoice details) load within 3 seconds.
- File uploads are initiated within 5 seconds, depending on file size and network conditions.
- These benchmarks are only guaranteed to work with a good internet connection (> 80 mbit/s, ping to our server < 150ms)
- They are also only tested and guaranteed on the following setup:
 - Chrome (guaranteed on current version: 146.0.76380.164)
 - Resolution: 1920x1080
 - OS: Windows 11
 - Min RAM: 8GB
 - Min CPU: Intel i7-8700k

NFR-2: Reliability and Data Integrity

Description

The system shall ensure that financial data is stored reliably and consistently without data loss or corruption.

Acceptance Criteria

- No invoice or payment record is stored in an incomplete or undefined state without clear status indication.
- In case of system errors, meaningful error messages are displayed to the user.
- If external services (e.g. Google Drive) fail, core data is still stored in the database.
- These criteria are only guaranteed under the following circumstances:
 - Stable power supply
 - Stable internet connection

NFR-3: Data Backup and Recovery

Description

All invoice receipts must be stored redundantly to prevent data loss.

Acceptance Criteria

- All uploaded invoices are stored both in the system database and in external storage (Google Drive)

NFR-4: Security

Description

The system shall protect sensitive financial and personal data from unauthorized access.

Acceptance Criteria

- Access to the system is restricted to authorized TUW Racing Team members via Google OAuth.
- Users can only access data that they are authorized to view.
- All communication with the system is encrypted (HTTPS).
- Input validation is enforced on the server side for all user inputs and file uploads. Additionally some validation is also already done on client-side. Main-validation does take place in the backend though.
- Uploaded files are only accessible to authorized users.

NFR-5: Privacy and Data Handling

Description

The system shall handle personal and financial data responsibly and minimize unnecessary data exposure.

Acceptance Criteria

- Only data required for invoice processing and payment handling is stored.
- Sensitive data (e.g. bank information, invoices) is not exposed to unauthorized users.
- Locally stored data (e.g. for offline functionality) is limited and clearly indicated to the user.

NFR-6: Usability

Description

The system shall be easy to use and minimize user effort and input errors.

Acceptance Criteria

- Core workflows (e.g. invoice submission) are intuitive and require minimal user training.
- Required input fields are clearly indicated.
- Validation errors are displayed clearly and help users correct mistakes.
- Users can easily understand the current status of invoices and payment requests.
- The system is optimized for a more efficient use compared to existing informal processes (e.g. email or messaging).
- This criteria can be evaluated by letting at least one member of the racing team test our application and give us their feedback.

NFR-7: Compatibility and Platform Support

Description

The system shall be usable across relevant platforms and devices.

Acceptance Criteria

- The system is optimized for desktop usage on:
 - Chrome (guaranteed on current version: 146.0.76380.164)
 - Resolution: 1920x1080
 - OS: Windows
- The system is optimized for mobile usage on:
 - Google Chrome (Android Version 16 on Samsung S24, UI Version 8.0, Resolution 360x780)
 - Safari (iOS 26.3.1 (a), iPhone 15, Resolution 393x852)
- Core functionality should be available on other platforms as well but this is not guaranteed
- Unsupported platform features (e.g. PWA limitations on iOS) could block essential workflows.

NFR-8: Maintainability

Description

The system shall be designed in a way that allows future changes and extensions with reasonable effort.

Acceptance Criteria

- The system follows a modular (service-oriented) architecture where components can be replaced or extended independently.
- The project can be set up and run locally using the provided configuration and documentation.
- The codebase follows consistent structure and conventions to support maintainability.
- State-of-the-art Linter will be used to ensure good code quality (automatically in pipeline as well)

NFR-9: Auditability and Traceability

Description

All relevant financial actions must be traceable for transparency and accountability.

Acceptance Criteria

- All status changes of invoices and payment requests are logged.
- Each log entry includes:
 - the user who performed the action
 - timestamp
 - previous and new status
 - optional reason
- The history of changes is accessible.

NFR-10: External Service Robustness

Description

The system shall remain functional even if external services are temporarily unavailable.

Acceptance Criteria

- Core functionality (e.g. invoice submission) remains available even if external services fail.
- Failures of external integrations (e.g. Google Drive, email, Slack) are handled gracefully.
- The system informs users or administrators about failed external operations when trying to access functionality which requires the external service.

NFR-11: File Upload Constraints

Description

The system shall support uploading invoice documents while preventing excessively large or unsupported files.

Acceptance Criteria

- Supported file formats include PDF, JPG and PNG.
- Maximum upload size per file is 20MB.
- Invalid file formats are rejected with a clear error message.

9 Domain Model

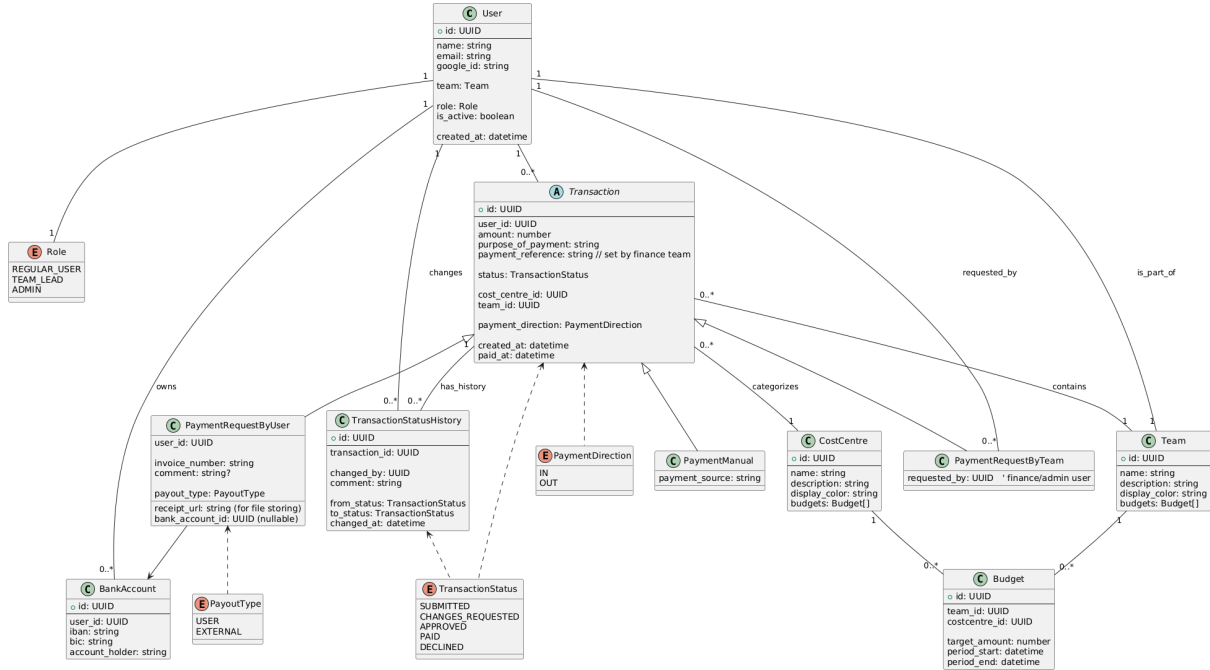


Figure 1: UML Diagram

10 Working Structure & Simple Project Plan

10.1 Distribution of Roles & Role Responsibilities

The details about the team-members and the roles that they fill in the project can be found in section 2. The following list concisely describes the responsibilities each team member has in their role:

- **Team Coordinator** (Tatzberger): Facilitates team meetings, tracks progress against milestones, manages the project contract and hand-in artefacts, serves as primary contact for the sponsor and course supervisors
- **Technical Architect** (Pelka): Defines the system architecture, technology stack decisions, API contracts between frontend and backend, and ensures architectural consistency across all components.
- **DevOps** (Lohwasser): Sets up and maintains CI/CD pipelines, containerisation (Docker), deployment environment, and developer tooling (linting, build scripts, environment configuration).
- **Test Lead** (Bräuer): Designs the test strategy, writes and maintains the test plan, coordinates unit/integration/system coverage, and tracks bugs.
- **UI Expert** (Jeschke): Owns the frontend design system, component library, user flows, and is responsible for usability of the interface across both Member and Finance/Admin roles.

- **AI Expert** (Schreiblehner): Leads design and implementation of AI-assisted features, defines AI-usage best-practices, evaluates model/library choices, and documents AI-related decisions.

However, this does not mean that the person that holds a specific role is the only one who is responsible for that part of the project. This is rather a position of expertise on which the other team-members can rely on in case they need help.

10.2 Project plan - PayTrack

1. Project Management
 - 1.1 Weekly team meetings & minutes
 - 1.2 Sponsor communication
 - 1.3 Contract & artefact submissions
2. Infrastructure & DevOps
 - 2.1 Repository setup & branching strategy
 - 2.2 setup development environment
 - 2.3 CI/CD pipeline (build , test , deploy)
3. Backend
 - 3.1 Authentication (Google OAuth, session management) [US-1]
 - 3.2 User & profile management [US-2, US-3, US-4, US-7]
 - 3.3 Invoice CRUD & lifecycle [US-5, US-6, US-7, US-9]
 - 3.4 Cost centre & team management [US-23, US-24, US-25, US-26]
 - 3.5 Payment management [US-9, US-10, US-11, US-12, US-18]
 - 3.6 Payment request management [US-13, US-14, US-15, US-16, US-17]
 - 3.7 Budget management & reporting [US-20, US-21, US-22]
 - 3.8 Exporting Data & Archiving [US-27, US-28]
 - 3.9 Slack/email notification integration [US-9, US-19]
4. Frontend
 - 5.1 Design system & component library
 - 5.2 User pages (profile , submittal form , ...) [US-2, US-5, US-7, US-14]
 - 5.3 Finance/Admin views (invoice list , approval flow , budgets , ...) [US-8, US-13, US-14, US-16, US-17, US-18, US-20, US-21, US-22]
 - 5.4 Settings & system management views [US-4, US-23, US-25]
5. Progressive Web App (PWA)
 - 5.1 PWA setup [US-29]
 - 5.2 Push-Notification integration [US-31]
 - 5.3 Offline mode [US-30, US-34]
 - 5.4 Optimize design & performance for mobile use [US-32, US-33]
6. Testing & QA
 - 6.1 Test plan document
 - 6.2 Unit tests (backend services)

6.3 Integration tests (API endpoints)

6.4 End-to-end tests (key user flows)

7. Documentation & Delivery

7.1 OpenAPI / REST API documentation

7.2 Domain model & architecture diagrams

7.3 Final project report & demo preparation

10.3 Milestones

Milestone	Milestone Meeting	Description
M1: Project Contract Submitted	MR1	Finalized and submitted project contract including asset architecture, Iceberg List, domain model draft, NFRs, and project plan.
M2: Infrastructure & Core Backend	IR1	Dev environment and CI/CD operational. Google OAuth login working. Core data models implemented. Invoice creation and lifecycle endpoints functional.
M3: Mostly functional Prototype	MR2	Most planned User-Stories implemented end-to-end (may have rough edges). Member and Finance/Admin flows fully navigable. Google Drive backup, Slack/email notifications, and bank statement upload integrated.
M4: Feature Complete Product	IR2	All User-Stories implemented. All critical bugs resolved. Performance and security review completed. Sponsor review / feedback round done.
M5: Final Delivery & Demo	MR3	Final artifact submission: source code, documentation, test report, final project report. Live demo prepared and delivered.

Table 1: Project Milestones

11 Project Boundaries

This section defines the scope of the PayTrack project by explicitly outlining which features and responsibilities are included and which are intentionally excluded. The goal is to ensure a clear understanding of the system’s purpose and to prevent scope creep during development.

11.1 In Scope

The following aspects are part of the PayTrack system and will be implemented within the project:

- A web-based application (including Progressive Web App capabilities) for managing invoices and payment requests within the TUW Racing Team.
- User authentication via Google OAuth and basic user profile management (including bank account information).
- Submission, validation, and tracking of invoices, including receipt uploads.
- A workflow for finance team members to review, approve, decline, and mark invoices as paid.
- Creation and management of payment requests (e.g. membership fees, event costs).
- Basic financial overview features, including filtering by teams, cost centres, and time periods.
- Import functionality for order lists (CSV) and bank statements with basic matching logic.
- Notifications via email (and optionally other integrations such as Slack).
- Export of financial data (e.g. CSV, PDF) for reporting purposes.
- Storage of invoice documents both within the system and in external storage (Google Drive).
- Mobile-friendly usage and support for common mobile and desktop browsers.

11.2 Out of Scope

The following aspects are explicitly not part of the PayTrack project:

- Integration with official accounting systems, tax software, or banking APIs beyond basic file import.
- Automated financial accounting, bookkeeping, or legally compliant tax processing.
- Guaranteed real-time synchronization across all components or external systems.
- Advanced AI-based processing with guaranteed accuracy (e.g. fully reliable invoice data extraction).
- Full cross-platform support for all browsers, devices, or operating systems.
- Enterprise-grade scalability, high-availability guarantees, or production-level service-level agreements (SLAs).
- Advanced security certifications or compliance with specific regulatory frameworks (e.g. GDPR certification processes, audits).
- Complete offline functionality of all system features (offline support is limited to selected use cases).
- User training, onboarding programs, or detailed documentation for non-technical stakeholders beyond basic usage.

- Long-term maintenance, hosting, or operational responsibility after project completion.
- Legal responsibility for accounting correctness, tax compliance, or financial reporting obligations.
- Support for multiple independent organizations or tenants using the same system instance.
- Advanced financial management features such as budgeting forecasts, multi-currency accounting, or financial analytics dashboards.
- Dynamic configuration of user roles or custom permission systems beyond the pre-defined roles.
- Advanced fraud detection, risk analysis, or financial anomaly detection.
- Development of fully native mobile applications for Android or iOS.
- Enterprise-level disaster recovery infrastructure or geographically distributed backups.

11.3 Assumptions and Constraints

The project is developed under the following assumptions and constraints:

- The system is designed for a relatively small user base (approx. 40–100 users).
- External services such as Google OAuth and Google Drive are available and functioning.
- The system is developed within the limited timeframe of a university course project.
- The application is intended to support internal processes of the TUW Racing Team and is not designed as a general-purpose financial platform.

12 Supply Components (Deliverables)

12.1 Software Artifacts

The result of the project for the customer will be the following, they will get a functioning:

- A web-based application which supports the functionalities described in section 7, while also adhering to the non-functional requirements detailed in section 8.
- A mobile application (app) that can be installed on mobile devices (i.e. phones) which also supports the full functionality that the web-application gives. Apart from this functionality, the features described in subsection 7.9 are also available in the mobile application.

The customer will receive the full source code for the developed software. The application has to be hosted by the customer themselves.

12.2 Management Artifacts

- Code documentation
- Meeting Minutes/Protocols
- Software architecture overview
- Test-plan

12.3 Out-of-scope Artifacts

- GANTT-Chart
- Marketing Materials

13 Risk Assessment

The project team uses a simple qualitative risk assessment based on two dimensions:

- **Probability (P):** likelihood that the risk occurs, rated from 1 (very low) to 5 (very high)
- **Impact (I):** severity of the consequences if the risk occurs, rated from 1 (very low) to 5 (very high)

The overall **risk score** is calculated as:

$$\text{Risk Score} = P \times I$$

Risks with a high score are monitored more closely and should be addressed early in the project through preventive measures and fallback plans.

13.1 Risk Register

Risk	Description	P	I	Score	Mitigation / Counter-measure
Unclear requirements from stakeholders	The finance workflow may not be fully documented at the beginning, which can lead to misunderstandings and wrong implementation priorities.	4	4	16	Conduct regular meetings with stakeholders, present implemented features to verify correctness, define clear user stories, and ensure that requirements are sufficiently considered, clarified, and agreed upon at an early stage.

Risk	Description	P	I	Score	Mitigation / Counter-measure
Expansion of project scope	Additional/unplanned features may be requested during development and exceed the available time.	2	3	6	Define a clear project scope and boundaries.
Low user adoption	Regular team members may avoid the platform if invoice submission is slower or more complicated than existing informal processes.	1	5	5	Focus on a simple and fast submission flow, test usability early, and gather feedback from representative end users before finalizing the UI.
Data privacy and sensitive financial information	The system processes invoices, payment-related information, and potentially personal data. Poor handling of this data may create compliance and trust issues.	3	5	15	Minimize stored personal data, use role-based access control, restrict visibility of financial records, and avoid storing unnecessary sensitive information.
Hosting issues	Problems with the hosting on reset, database setup, or external services may delay testing and final delivery.	3	4	12	Set up infrastructure early, prepare a deployment pipeline, test the hosting environment continuously.
Client Deployment	Uncertain deployment environment of the client can cause a major time shift.	4	3	12	Specify the client deployment flow early and continuously cross-deploy it on reset and client.
Insufficient testing	Errors in payment tracking, invoice status handling, or validation logic may remain undetected and reduce reliability.	2	5	10	Proper test plan and test coordinator needs to coordinate the testing responsibilities and overlook the test coverage.
Team member availability	Because this is a student project, exams, deadlines, or illness may reduce the availability of key contributors.	4	3	12	Distribute knowledge across the team, define backup roles, document technical decisions, and review progress frequently to detect bottlenecks early.

Risk	Description	P	I	Score	Mitigation / Counter-measure
Underestimation of technical complexity	Features such as PWA support or AI-related functionality may require more effort than initially expected or defined user stories may not be possible for some features due to technical restrictions of IOS devices.	4	4	16	Identify technically critical features early in the planning phase, discuss platform-specific limitations such as iOS restrictions before implementation, and prioritize core functionality within the available time frame. Where technical constraints prevent full implementation, adjust the affected user stories in coordination with stakeholders and reduce non-essential scope.
Security vulnerabilities	Weak authentication, insufficient authorization checks, or insecure file handling could expose financial data or allow unauthorized actions.	2	5	10	Apply secure authentication and authorization principles, validate uploaded files, restrict access based on roles, and review security-critical functionality carefully.

13.2 Most Critical Risks

The most critical risks for the PayTrack project are *Unclear requirements from stakeholders* and *Underestimation of technical complexity*, both with a risk score of 16. These risks are particularly significant because they directly affect whether the implemented solution matches stakeholder expectations and whether important features can be delivered within the available time frame.

The risk *Data privacy and sensitive financial information*, with a score of 15, is also highly relevant. Since the system processes invoices and payment-related information, improper handling of such data could reduce trust in the platform and lead to serious security and compliance concerns.

In addition, risks such as *Hosting issues*, *Client Deployment*, and *Team member availability* remain important because they may delay testing, deployment, and delivery even if the software itself is implemented correctly.

In particular, this risk is increased by the limited project schedule, as less than one week is effectively available for exploring and validating technically demanding features. Under these conditions, dedicated prototyping is largely infeasible, which makes early feasibility assessment and strict prioritization even more important.

13.3 Risk Handling Strategy

The project team manages risks through continuous monitoring and early coordination. Risks are reviewed regularly during meetings and are reassessed whenever major technical or organizational changes occur. Special attention is given to risks with high probability or high impact.

To reduce project risk, the team focuses on clear communication with stakeholders, early clarification of requirements, and the prioritization of core functionality. Technical constraints, especially platform-specific limitations such as iOS restrictions, are considered early in planning so that affected user stories can be adapted if necessary.

Furthermore, responsibilities within the team are distributed clearly to avoid knowledge concentration, and testing activities are coordinated through a defined test plan. Deployment-related risks are addressed by aligning the hosting and client deployment process as early as possible and validating the setup continuously throughout the project.

13.4 Risk Matrix

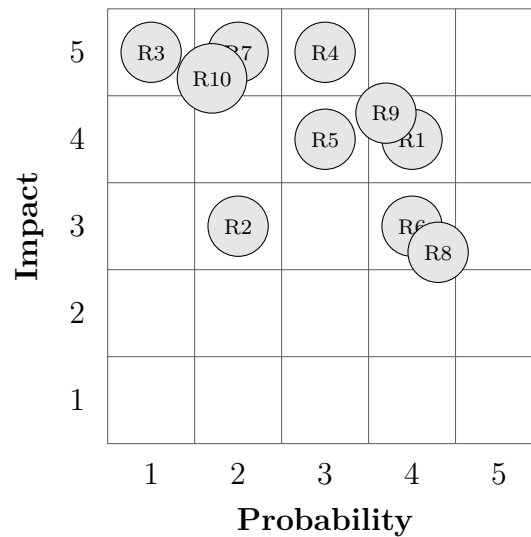


Figure 2: Risk matrix for the PayTrack project.

Legend:

- R1: Unclear requirements from stakeholders
- R2: Expansion of project scope
- R3: Low user adoption
- R4: Data privacy and sensitive financial information
- R5: Hosting issues
- R6: Client deployment
- R7: Insufficient testing
- R8: Team member availability

- R9: Underestimation of technical complexity
- R10: Security vulnerabilities

14 Information Process

14.1 Communication

Internal Communication: The internal communication will mainly take place on a Discord-Server. Threads are created to track communication for each specific issue (e.g. a specific feature to be implemented). Within these threads, discussions, technical documents, links, etc. are shared which concern the issue in order to have clear and traceable communication for each topic.

The Discord-Server also has voice-channels, where direct discussion can be held between members in order to make communication more efficient and less time-consuming.

For other topics, or when a reply is needed quickly, a Whatsapp group is also present where non-technical/implementation relevant discussions are held.

External Communication: The external communication is also split into two channels. Communication with the TUW-Racing Team also happens via the aforementioned Discord-Server, however the involved TUW-Racing members have limited permissions on the server and can not see all discussions.

Communication with the Tutor (e.g. to arrange meetings) or other members directly involved in the course is handled via E-Mail.

14.2 Documentation

Meetings: Official Meetings and Meetings where the whole team (and/or the Tutor) is involved will be documented using meeting notes. One of the participants takes notes during the discussion in order to capture the following points:

- Start/End-Time of the Meeting
- Participants
- Who is absent (and why)
- The goal of the meeting
- Decisions that are made during the meeting (and why they were decided that way)

These meeting notes are stored in the GitLab Wiki.

Unofficial meetings, where only a small part of the team members are present (e.g. to discuss some problem/help each other) will not be documented. However, if some relevant changes are proposed in such a meeting, they are posted in the relevant thread on Discord to let the whole team know/discuss it.

Time-Tracking: To track the time that is spent on the project/specific features, issues are created on GitLab, where the used time can be spent. Each project-member is responsible for correctly tracking the time they spent.

15 Special Features

The PayTrack project is developed in close collaboration with the TUW Racing Team, which acts as a real stakeholder by providing domain knowledge, feedback on requirements, and validation of implemented features. This ensures that the system is aligned with actual workflows and practical needs. Additionally, there is the possibility that the application will be hosted within the infrastructure of the TUW Racing Team. However, this deployment approach is not yet fully finalized, as the feasibility and effort required for integration into the existing environment are still being evaluated. This uncertainty is considered during development to ensure that alternative deployment options remain possible if required.

Furthermore, the application relies on external services such as Google OAuth for authentication and Google Drive for document storage, introducing dependencies that are outside the direct control of the development team.